



COLLEGE CODE : 8301

**COLLEGE NAME : GOVERNMENT COLLEGE OF
ENGINEERING, SRIRANGAM**

DEPARTMENT : CSE

STUDENT NM-ID :

AFCB7FD1D6BFB2687274627EA25C776F

ROLL NO : 830123104706

DATE : 08-09-2026

Name : AI E-Commerce Customer Support Agent

SUBMITTED BY

Aarthi Babu

MOBILE: 6374460114

Table of Contents

- . Introduction
- . Project Setup
- . Core Features
Implementation
- . Data Storage (Local State /
Database)
- . Testing Core Features
- . Version Control (GitHub)
- . Conclusion

Introduction

Modern e-commerce platforms receive a large number of customer requests related to products, orders, returns, and recommendations. Customers expect quick and accurate responses without having to search through several pages or wait for a support executive. An AI E-Commerce Customer Support Agent can provide a conversational interface that understands customer questions and assists them throughout the shopping journey.

The AI E-Commerce Customer Support Agent project aims to build an intelligent support system that can answer product-related questions, check order status, assist with return requests, and provide relevant product recommendations. The agent combines conversational AI with tool calling so that it can access the required application functions instead of responding only from static knowledge.

The system is designed around four major customer-support activities: product information, order-status assistance, returns support, and personalized recommendations. Product queries can be answered from the available catalog, while order and return requests can be handled by calling appropriate application tools. Recommendation responses can use the customer's current conversation and remembered preferences to make the interaction more useful.

Memory is an important capability of the agent because customers may express preferences such as product category, size, budget, or previously discussed

requirements. Remembering useful non-sensitive preferences allows the agent to avoid repeatedly asking the same questions and provide more relevant assistance in later interactions.

This report documents the development of the minimum viable product (MVP), including project setup, core agent capabilities, data storage and memory, testing, and version control. The goal is to demonstrate how an AI agent can combine natural-language interaction with practical e-commerce tools to deliver a complete customer-support experience.

The project therefore focuses not only on generating natural-language answers but also on selecting the right action for each customer request, obtaining information from the appropriate tool, and presenting the result clearly. This approach makes the support assistant more useful for real-world e-commerce workflows.

Phase:



MVP Implementation

1. Project Setup

The initial phase of the AI E-Commerce Customer Support Agent involved defining the customer-support workflow and preparing a modular application structure. The MVP was planned around a conversational frontend, an agent/backend layer, a product and order

data layer, and a set of tools that the agent can call when a request requires live application information.

A web application architecture can use React for the customer-facing interface and a Node.js/Express backend for APIs and agent orchestration. A database such as MongoDB can store product, customer, and order information. This structure separates the user interface from business operations and makes it easier to extend the agent with additional tools in future versions.

The project setup also requires an AI model connection and a tool-calling mechanism. The agent is given clear tool descriptions and rules so it can decide when to search products, retrieve an order, check return eligibility, or obtain recommendation data. Environment variables are used for configuration values and API credentials rather than hard-coding them into the application.

The frontend provides a chat interface where users can enter natural-language requests and view the agent's responses. Backend API routes coordinate requests between the chat interface, the AI model, application tools, and stored data. The product catalog and order records are represented in structured form so that tool results can be returned consistently to the agent.

Version control is configured with Git and a GitHub repository so that changes to the agent, frontend, backend, and data models can be tracked. This setup creates a foundation for testing, debugging, collaborative development, and later deployment of the customer-support application.

Version control was implemented using Git, with a remote repository hosted on GitHub, allowing systematic tracking of changes and enabling collaboration. This setup provided a robust foundation, facilitating smooth development, debugging, and deployment workflows throughout the project lifecycle.

2. Core Features Implementation

The core functionality of the system is implemented as an AI-powered customer-support workflow. The user communicates through a chat interface, while the agent identifies the intent of each request and decides whether a direct answer is sufficient or a tool must be called. The major capabilities are product queries, order-status requests, returns assistance, recommendations, and conversational memory.

For product queries, the agent can call a product-search or catalog tool using details provided by the customer, such as product name, category, price range, or required features.

The tool returns matching product information, which the agent summarizes in a simple conversational response.

This reduces the need for customers to manually search through the catalog.

For order-status requests, the agent can request the necessary order information through an order-lookup tool. After receiving the tool result, it can communicate the current order state, such as processing, shipped, or delivered, together with relevant details available to the application. For return-related requests, a return-eligibility tool can check the applicable order information and provide the next steps.



The recommendation capability uses available catalog information together with conversation context and remembered preferences. For example, if a customer has already mentioned a preferred category or budget, the agent can use that context when producing recommendations. Tool calling and memory work together: tools provide current application data, while memory helps maintain useful context across interactions.

3. Data Storage (Local State / Database)

Data persistence for the AI E-Commerce Customer Support Agent can be organized around product, customer, order, and return records. A document-oriented database such as MongoDB is suitable for an MVP because product and order objects can be stored in flexible document structures and accessed through backend APIs.

The product collection can contain fields such as product identifier, name, category, description, price, availability, and relevant attributes. Order records can contain an order identifier, customer reference, item information, status, purchase date, and return-related information. These structured records allow the agent's tools to retrieve precise information instead of depending only on generated text.

The backend provides controlled functions for reading and updating application data. Tool definitions expose only the operations required by the support agent, such as product search, order lookup, return checking, and recommendation retrieval. Validation is applied to tool inputs so that invalid requests do not directly reach the

database operations.

Conversational memory is maintained separately from the core transaction data. Short-term memory can preserve the current conversation, while longer-term memory can store useful customer preferences when the application is designed to support them. Examples include preferred categories, approximate budget, or previously discussed product requirements. Sensitive information should not be stored unnecessarily.

This combination of structured application data, controlled tools, and conversational memory provides both accuracy and continuity. The agent can retrieve current order or catalog information through tools while maintaining enough context to make the conversation feel consistent and personalized.

4. Testing Core Features

Testing the customer-support agent focuses on both conversational quality and functional correctness. The main scenarios include product searches, order-status questions, return requests, recommendations, invalid requests, and follow-up questions that depend on earlier conversation context.

Unit testing can be used for individual backend functions such as product search, order lookup, return-eligibility checks, recommendation logic, and memory operations. Integration testing verifies that the chat interface, agent layer, tools, and database communicate correctly. Test cases should confirm that the agent selects the correct tool and uses the returned information in its response.

Manual testing can be performed through the web interface and API testing tools such as Postman. Example scenarios include asking for a product under a specified budget, checking an existing order, asking whether an item can be returned, and requesting recommendations after stating a preference. Edge cases should also be tested, such as unknown order IDs, unavailable products, incomplete requests, and

unsupported actions.

Memory-related tests verify that useful preferences are retained when intended and that irrelevant or sensitive information is not unnecessarily stored. Tool-failure tests verify that the agent responds safely when a product service, order lookup, or database operation is unavailable. These tests help ensure that the system remains understandable and reliable under realistic customer interactions.

The testing process should confirm that the agent provides clear responses, calls tools only when needed, handles failures gracefully, and maintains the expected conversation context.

5. Version Control (GitHub)

Version control is an important part of the development lifecycle of the AI E-Commerce Customer Support Agent. A GitHub repository can be used as the central location for source code, documentation, configuration templates, and project history. This makes development changes traceable and supports collaboration between team members.

A practical branch structure can separate stable code from development work and individual features. Feature branches can be used for tasks such as the chat interface, product-search tool, order-status tool, return workflow, recommendation module, and memory functionality. Changes can be reviewed before being merged into the main branch.

Commit messages and issue tracking help document improvements and defects. The repository can also contain a README describing project setup, environment variables, application execution, available tools, test cases, and expected workflows. Keeping credentials outside the repository is essential when connecting the application to external AI or database services.

Conclusion

The AI E-Commerce Customer Support Agent demonstrates how an AI agent can be combined with practical application tools to support common online-shopping activities. Instead of providing only general conversational answers, the agent can use product, order, return, and recommendation tools to respond with application-specific information.

The project also demonstrates the role of memory in creating a continuous customer experience.

Remembered preferences and conversation context can help the agent provide more relevant recommendations and reduce repetitive questions, while structured database records provide the information required for accurate transactional responses.

Testing across product queries, order tracking, returns, recommendations, tool failures, and memory scenarios helps verify the reliability of the MVP. Version control through GitHub provides a disciplined method for maintaining the project and documenting changes throughout development.

Future enhancements can include richer product search, multilingual customer support, improved recommendation models, voice-based interaction, analytics dashboards, human-agent handoff, notification integration, and stronger access controls.

Additional safeguards can also be introduced to manage customer data responsibly and improve the reliability of tool-based actions.

Overall, the project provides a practical blueprint for an AI-powered e-commerce support system in which natural-language conversation, tool calling, structured application data, and memory work together to improve the customer experience.